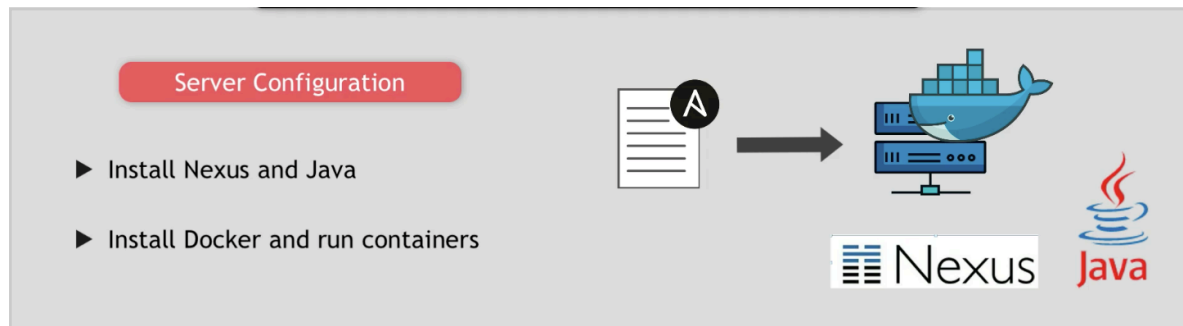


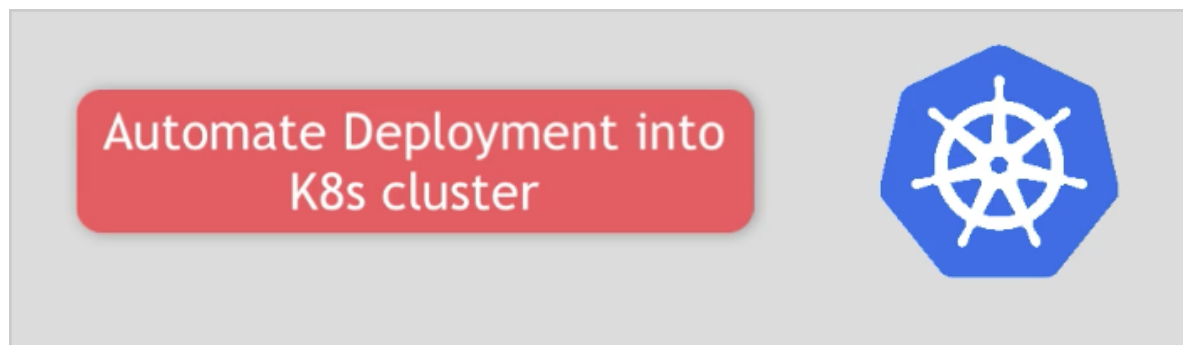
Module 15 - chapter 20

Deploying Application in K8s

So far we have configure servers directly using ansible



In this chapter we will use terraform to deploy an application into a Kubernetes Cluster

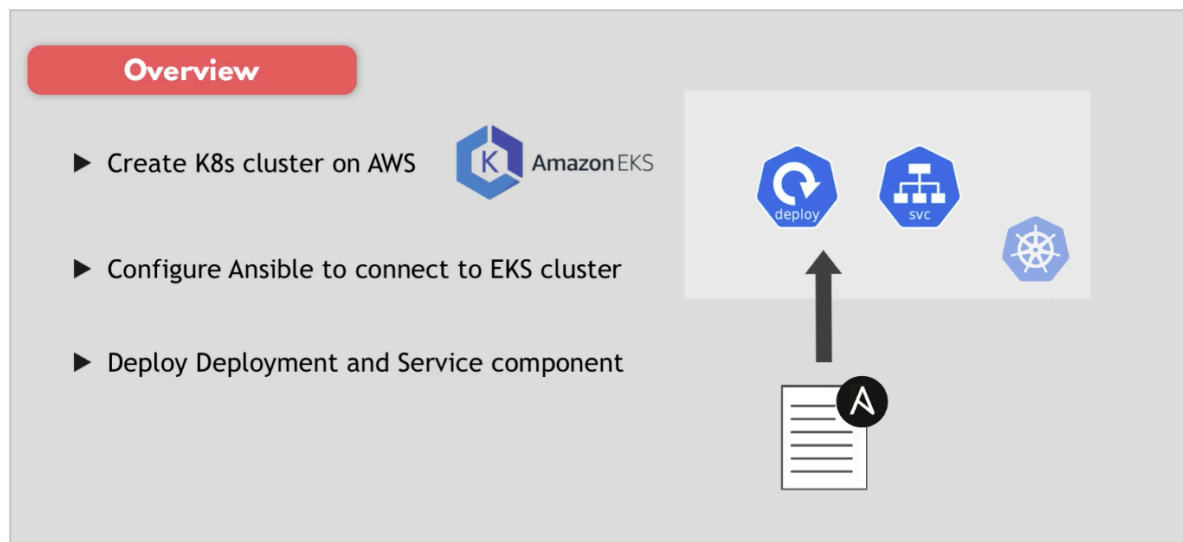


Overview

- Create a K8s cluster in AWS using Terraform (Module 12 - chapter 19 to 21)



- Then we will configure ansible to connect to the EKS cluster and deploy a simple deployment (deploy) and service component (svc) inside the cluster



Create EKS cluster with Terraform

I use the branch from Module 12 - chapter 21

https://github.com/AstridCaballero/Module_12_Terraform/blob/Module_12/Terraform_chapter-21/eks-cluster.tf

-> terraform init

The screenshot shows a code editor with the Terraform configuration file `eks-cluster.tf` open. The configuration defines an EKS cluster named `myapp-eks-cluster` using the `terraform-aws-modules/eks/aws` module. The configuration includes settings for the cluster name, Kubernetes version, subnet IDs, VPC ID, and various add-ons like `coredns`, `eks-pod-identity-agent`, `kube-proxy`, and `vpc-cni`. The `terraform init` command has been executed, and the output shows that the modules have been initialized successfully.

```
terraform -> eks-cluster.tf
1 module "eks" {
2   source = "terraform-aws-modules/eks/aws"
3   version = "21.23.0"
4
5   name = "myapp-eks-cluster"
6   kubernetes_version = "1.36"
7
8   subnet_ids = module.myapp-vpc.private_subnets
9   vpc_id      = module.myapp-vpc.vpc_id
10
11   endpoint_public_access = true
12   enable_cluster_creator_admin_permissions = true
13
14   addons = {
15     coredns = {}
16     eks-pod-identity-agent = {
17       before_compute = true
18     }
19     kube-proxy = {}
20     vpc-cni = {
21       before_compute = true
22     }
23   }
24 }
```

Terminal: zsh x +
\$ terraform init
Initializing modules...
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Reusing previous version of hashicorp/tls from the dependency lock file
- Reusing previous version of hashicorp/time from the dependency lock file

-> terraform apply

```
module.eks.aws_eks_addon.this["coredns"]: Creation complete after 1m 10s
module.eks.aws_eks_addon.this["kube-proxy"]: Still creating... [00m 00s]
module.eks.aws_eks_addon.this["kube-proxy"]: Creation complete after 1m 10s

Apply complete! Resources: 62 added, 0 changed, 0 destroyed.
```

~/terraform ➔ Module_12/Terraform_chapter-21 !1 ➔

Cluster name	Status	Kubernetes version	Support period	Upgrade policy	Created	Provider
myapp-eks-cluster	Active	1.36	Standard support until August 2, 2027	Extended support	11 minutes ago	EKS

Is active and we have 3 nodes

Node name	Instance type	Compute	Managed by	Created	Status	CPU usage	Memory usage	Ephemeral storage usage
ip-10-0-1-107.eu-west-2.compute.internal	t3.small	Node group	dev-20260812105436185300000000	2 minutes ago	Ready	12%	25%	15%
ip-10-0-2-30.eu-west-2.compute.internal	t3.small	Node group	dev-20260812105436185300000000	2 minutes ago	Ready	17%	29%	15%
ip-10-0-3-33.eu-west-2.compute.internal	t3.small	Node group	dev-20260812105436185300000000	2 minutes ago	Ready	17%	29%	15%

Now with ansible we want to connect to this cluster and deploy a very simple 'deployment' and 'service' component into that cluster.

Create a Namespace in EKS cluster

We need Kubeconfig file to be able to connect to our cluster see Module 11 - chapter 2, Module 12 - chapter 21 and '.kube/config file and setting the context' notes:

Module 12 - chapter 21
25/06/2026 ...eks update-kubeconfig --name myapp-e-...
DevOps_Nano — iCloud

Module 11 - chapter 2
25/06/2026 ...eks update-kubeconfig --name eks-clust...
DevOps_Nano — iCloud

.kube/config file and setting the context
12/06/2026 ...eks update-kubeconfig: aws eks update-...
DevOps_Nano — iCloud

Attachments

% aws eks update-kubeconfig --name eks-Cluster-test
PNG Image · 79 KB

1. Configure our environment so kubectl can connect to our cluster
-> aws eks update-kubeconfig --name myapp-eks-cluster --region eu-west-2

The above will update our kubeconfig file, so it has the correct ca cert (ca certificate), token, IAM authenticator, etc)

The kubeconfig file contains all the information we need for the connection like : server address, the host name, certificates, etc.

So ansible will use the kubeconfig information to connect to the new EKS.

I check ~/.kube/config -> default config file
-> cat ~/.kube/config

And the current-context is

```
name: arn:aws:eks:eu-west-2:706976333940:cluster/eks-cluster-test
- context:
  cluster: arn:aws:eks:eu-west-2:706976333940:cluster/myapp-eks-cluster
  user: arn:aws:eks:eu-west-2:706976333940:cluster/myapp-eks-cluster
  name: arn:aws:eks:eu-west-2:706976333940:cluster/myapp-eks-cluster
current-context: arn:aws:eks:eu-west-2:706976333940:cluster/myapp-eks-cluster
kind: Config
users:
- name: arn:aws:eks:eu-west-2:706976333940:cluster/eks-cluster-test
  user:
    exec:
```

And we compare with AWS console info and it is the same

myapp-eks-cluster Connect Actions Monitor cluster

Cluster info

Status Active	Kubernetes version 1.36	Support period Standard support until August 2, 2027	Provider EKS
Cluster health 0	Upgrade insights 5	Node health issues 0	Capability issues 0

Overview | **Updated** | Resources | Compute | Networking | Add-ons | Capabilities | Access | Observability | Update history & backups | Tags

Details

API server endpoint https://69DC848BFBE791941D1D1F3AA6063DF.gr7.eu-west-2.eks.amazonaws.com	OpenID Connect provider URL https://oidc.eks.eu-west-2.amazonaws.com/id/69DC848BFBE791941D1D1F3AA6063DF	Created 19 minutes ago
Certificate authority LS0LS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSUREakNDQWZ hZ0F3SUJBZ0lSQU9udEFZZitvSFVW50ZmhtPDIzaTUV3RFFZSk vWklodmNOQVFFTEJRQXcKXIRFVE1CRUdBMVVFQXhNS2EzVmla	Cluster IAM role ARN arn:aws:iam::706976333940:role/myapp-eks-cluster-cluster-20260812 104546326200000002 View in IAM	Cluster ARN arn:aws:eks:eu-west-2:706976333940:cluster/myapp-eks-cluster
Platform version eks.10		

So I don't need to update the default kube config file but Nana creates a new config file specifically for this project so I will do that:

-> `aws eks update-kubeconfig --region eu-west-2 --name myapp-eks-cluster --kubeconfig ~/terraform/kubeconfig_myapp-eks-cluster`

`--kubeconfig ~/terraform/kubeconfig_myapp-eks-cluster` -> it creates the new kubeconfig file in the terraform project and it calls it 'kubeconfig_myapp-eks-cluster'

```

eks-cluster.tf
28 instance_types = ["t3.small"]
29
30 min_size      = 1
31 max_size      = 3
32 desired_size  = 3
33
34 }
35
36 tags = {
37   "Environment" = "dev"
38   application    = "myapp"
39 }
40

```

```

module.eks.aws_eks_addon.this["kube-proxy"]: Creating...
module.eks.aws_eks_addon.this["coredns"]: Creating...
module.eks.aws_eks_addon.this["kube-proxy"]: Still creating... [00m10s elapsed]
module.eks.aws_eks_addon.this["coredns"]: Still creating... [00m10s elapsed]
module.eks.aws_eks_addon.this["coredns"]: Creation complete after 14s [id=myapp-eks-cluster:coredns]
module.eks.aws_eks_addon.this["kube-proxy"]: Still creating... [00m20s elapsed]
module.eks.aws_eks_addon.this["kube-proxy"]: Creation complete after 24s [id=myapp-eks-cluster:kube-proxy]

Apply complete! Resources: 62 added, 0 changed, 0 destroyed.

~/terraform Module_12/Terraform_chapter-21_12 $ aws eks update-kubeconfig --region eu-west-2 --name myapp-eks-cluster --kubeconfig ~/t
erraform/kubeconfig_myapp-eks-cluster

```

And I run it and now we can see the new kube config file in the project tree:

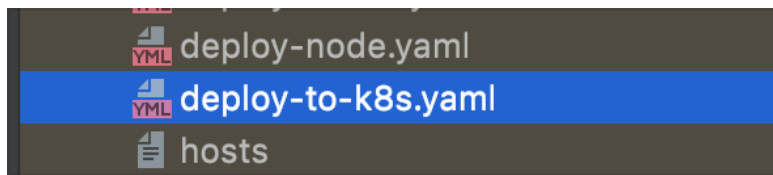
The screenshot shows a VS Code editor with a project named 'kubernetes_myapp-eks-cluster'. The file explorer on the left shows a directory structure with files like 'providers.tf', 'eks-cluster.tf', and 'terraform.tfvars'. The main editor window shows the 'eks-cluster.tf' file with Terraform code for an AWS EKS cluster. The terminal at the bottom shows the output of a Terraform apply command, indicating that resources were created successfully. The terminal output includes messages like 'module.eks.aws_eks_addon.this["kube-proxy"]: Creating...', 'module.eks.aws_eks_addon.this["coredns"]: Creating...', and 'Apply complete! Resources: 62 added, 0 changed, 0 destroyed.'

Now we have the contents of the kubeconfig file locally.

The screenshot shows a VS Code editor with a project named 'kubernetes_myapp-eks-cluster'. The file explorer on the left shows a directory structure with files like 'providers.tf', 'eks-cluster.tf', and 'kubernetes_myapp-eks-cluster'. The main editor window shows the 'kubernetes_myapp-eks-cluster' file with the contents of a kubeconfig file. The terminal at the bottom shows the output of a Terraform apply command, indicating that resources were created successfully. The terminal output includes messages like 'module.eks.aws_eks_addon.this["kube-proxy"]: Creating...', 'module.eks.aws_eks_addon.this["coredns"]: Creating...', and 'Apply complete! Resources: 62 added, 0 changed, 0 destroyed.'

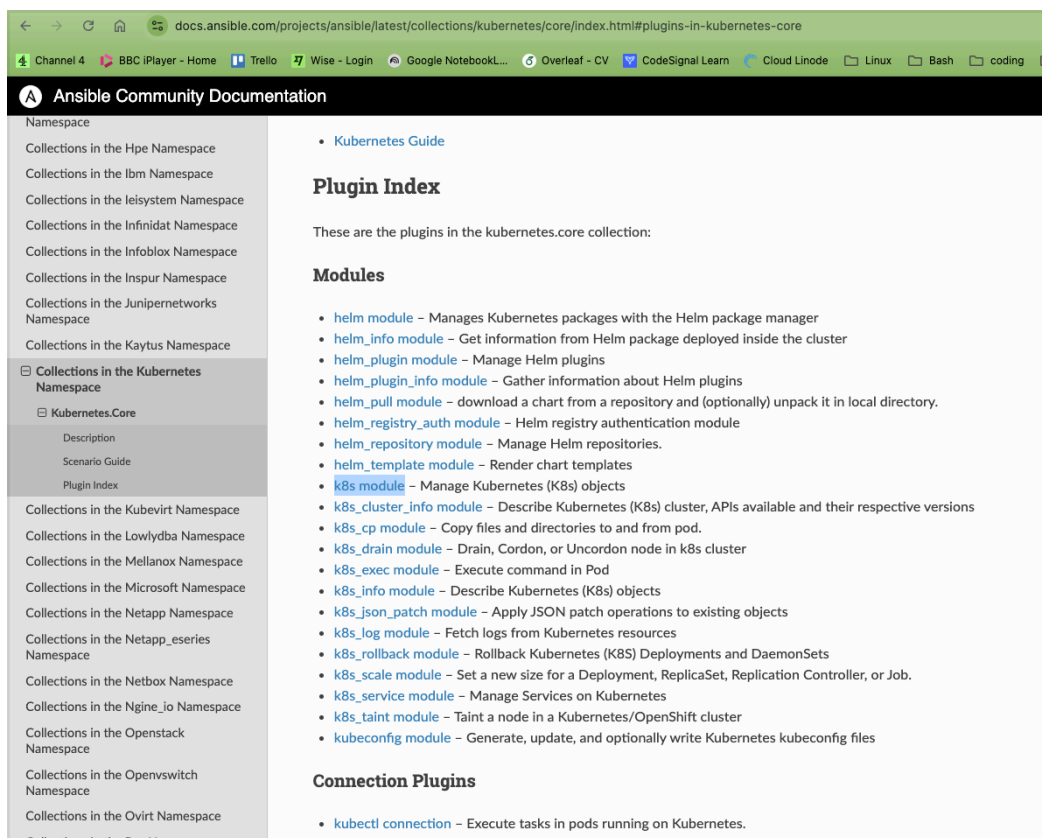
And now ansible can use that information!

Now we go to our ansible project and create a new yaml file called 'deploy-to-k8s.yaml'



Now we will write an ansible play to deploy an app into K8s cluster, and for that there is a dedicated ansible module from Kubernetes

<https://docs.ansible.com/projects/ansible/latest/collections/kubernetes/core/index.html#plugins-in-kubernetes-core>



And it is called 'k8s' module

https://docs.ansible.com/projects/ansible/latest/collections/kubernetes/core/k8s_module.html#ansible-collections-kubernetes-core-k8s-module

docs.ansible.com/projects/ansible/latest/collections/kubernetes/core/k8s_module.html#ansible-collections-kubernetes-core-k8s-module

Channel 4 BBC iPlayer - Home Trello Wise - Login Google NotebookLM Overleaf - CV CodeSignal Learn Cloud Linode Linux Bash coding Java DevOps

Ansible Community Documentation

Collections in the IBM Namespace
Collections in the leisystem Namespace
Collections in the Infinidat Namespace
Collections in the Infoblox Namespace
Collections in the Inspur Namespace
Collections in the Juniper Networks Namespace
Collections in the Kaytus Namespace

Collections in the Kubernetes Namespace

- Kubernetes.Core**
 - Description
 - Scenario Guide
 - Plugin Index

Collections in the Kubevirt Namespace
Collections in the Lowlydba Namespace
Collections in the Mellanox Namespace
Collections in the Microsoft Namespace
Collections in the Netapp Namespace
Collections in the Netapp_eseries Namespace
Collections in the Netbox Namespace
Collections in the Nginx.io Namespace
Collections in the Openstack Namespace
Collections in the Openswitch Namespace
Collections in the Ovirt Namespace
Collections in the Pcg Namespace
Collections in the Purestorage

[Collection Index](#) / [Collections in the Kubernetes Namespace](#) / [Kubernetes.Core](#) / [kubernetes.core.k8s module](#) - Manage Kubernetes (K8s) objects

This is the **latest** (stable) Ansible community documentation. For Red Hat Ansible Automation Platform subscriptions, see [Life Cycle](#) for version information.

kubernetes.core.k8s module – Manage Kubernetes (K8s) objects

Note

This module is part of the [kubernetes.core](#) collection (version 6.5.0).

You might already have this collection installed if you are using the [ansible](#) package. It is not included in [ansible-core](#). To check whether it is installed, use `ansible-galaxy collection list kubernetes.core`. To check whether it is installed, use `ansible-galaxy collection install kubernetes.core`. You need further requirements to be able to use this module, see [Requirements](#).

To use it in a playbook, specify: `kubernetes.core.k8s`.

- [Synopsis](#)
- [Requirements](#)
- [Parameters](#)
- [Notes](#)
- [Examples](#)
- [Return Values](#)

Synopsis

- Use the Kubernetes Python client to perform CRUD operations on K8s objects.
- Pass the object definition from a source file or inline. See examples for reading files and using Jinja templates or vault-encrypted files.
- Access to the full range of K8s APIs.
- Use the [kubernetes.core.k8s_info](#) module to obtain a list of items about an object of type `kind`.
- Authenticate using either a config file, certificates, password or token.
- Supports check mode.

This k8s ansible module makes it easy to create components inside Kubernetes, to delete them, to configure things in there.

So instead of running kubectl commands or even installing kubectl we can use this k8s module to connect to the EKS cluster and do anything we want inside there.

To test how the module works we will create a Namespace and here is the documentation example:

Examples

```
- name: Create a k8s namespace
  kubernetes.core.k8s:
    name: testing
    api_version: v1
    kind: Namespace
    state: present

- name: Create a Service object from an inline definition
  kubernetes.core.k8s:
    state: present
```

And I copy the example and paste it in my play:

```
1  ---
2  - name: Deploy app in new namespace
3    tasks:
4  - name: Create a k8s namespace
5    kubernetes.core.k8s:
6      name: testing
7      api_version: v1
8      kind: Namespace
9      state: present
```

In Module 10 - chapter 8 Namespaces - Organizing Components we learn't how to create Kubernetes Namespaces

The screenshot shows a Kubernetes dashboard interface. On the left, there's a sidebar with 'Attachments' including various images and documents. The main area displays a terminal window with the following output:

```
kube-system    Active    2d16h
my-namespace    Active    40s
astrid@192.168.1.183 /Users/astrid
%
```

Below the terminal, a message states: 'Another way to create a namespace is to use a configuration file -> better!'. This is followed by a diagram titled '4 Namespaces per Default' showing a blue hexagon with 'ns' inside. Below the diagram, the text reads: 'Create a namespace with a configuration file'. To the right, a code block shows the following YAML configuration:

```
apiVersion: v1
kind: Namespace
metadata:
  name: my-namespace
```

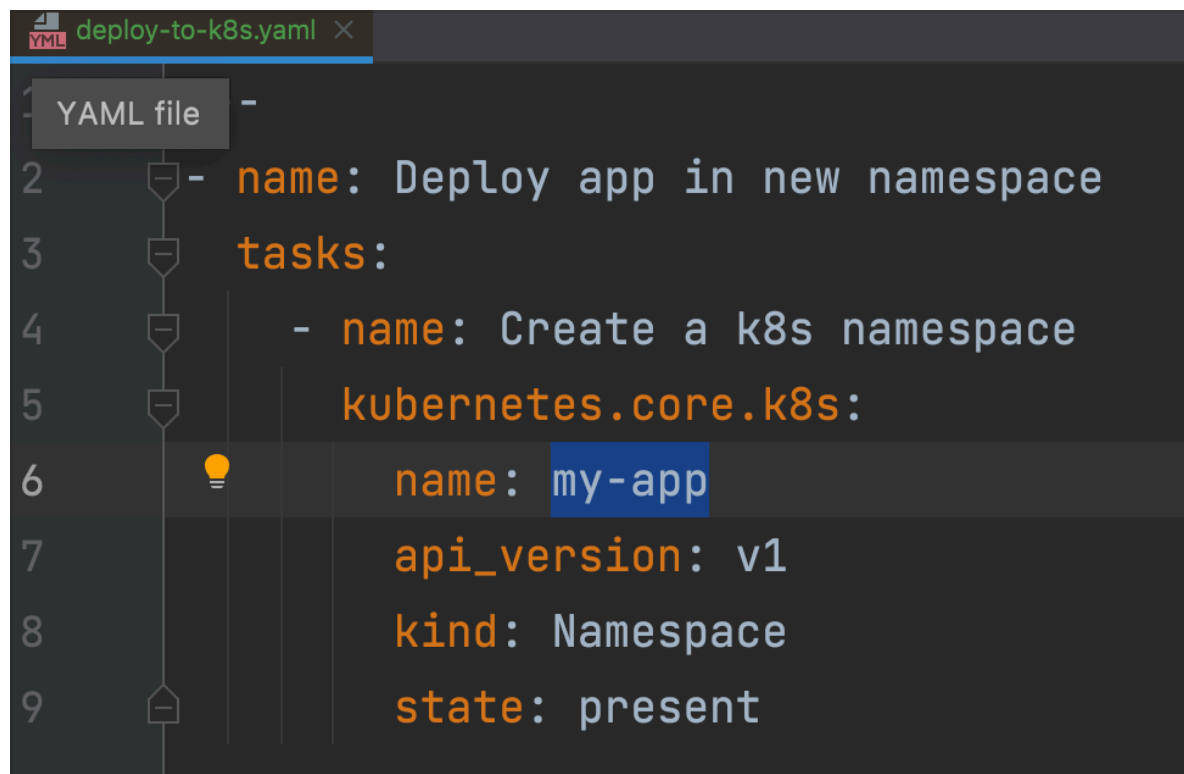
At the bottom, there's a section titled 'Why namespace?' with the question 'When to use them?' and a list item: '1. All the resources and everything are in one namespace, let's say default.'

And we can see that the Kubernetes yaml has the same attribute names as the ansible k8s module:

```
api_version: v1
kind: Namespace
```

So we can take the configuration from the Kubernetes file and copy them here in the ansible play. And we could write all the k8s configuration files directly in our ansible play if we want and don't want to have a separate k8s configuration file.

And let's name the namespace 'my-app'



```
YAML file  deploy-to-k8s.yaml x
1  -
2  - name: Deploy app in new namespace
3    tasks:
4      - name: Create a k8s namespace
5        kubernetes.core.k8s:
6          name: my-app
7          api_version: v1
8          kind: Namespace
9          state: present
```

Now let's add the 'hosts' attribute with the value 'localhost' in order to execute ansible locally

```
YML deploy-to-k8s.yaml x
1  ---
2  - name: Deploy app in new namespace
3    hosts: localhost
4    tasks:
5      - name: Create a k8s namespace
6        kubernetes.core.k8s:
7          name: my-app
8          api_version: v1
9          kind: Namespace
10         state: present
```

And by running ansible locally it will fetch the local kube config file to connect to the EKS cluster.

So we don't need to run the ansible play inside a server, so we don't need the inventory and their IP addresses. We will run the ansible k8s module locally in my laptop.

—— tangent hosts: localhost ——

Why hosts: localhost is correct here

Ansible's hosts: field controls where the Ansible module code executes, not what it operates on. The kubernetes.core.k8s module doesn't SSH anywhere or run shell commands on a remote box — it's a Python module that makes HTTPS API calls to the Kubernetes API server, using the kubernetes Python client library.

So the "host" you're targeting is really just: which machine should run this Python process and make the API call? For any cloud-API-driven module (AWS, Kubernetes, Azure, etc.), that's almost always wherever Ansible itself is running — i.e. localhost — unless you have a reason to proxy the API call through a bastion or specific machine.

Contrast this with something like ansible.builtin.service, which really does need to SSH into a remote host and run systemctl there. k8s isn't that kind of module.

How Ansible "knows the host" (localhost)

Since localhost isn't in your inventory (or even if it is, implicitly), Ansible has a built-in implicit localhost entry with connection: local — meaning "just run this in the current process/shell, no SSH." No inventory file entry is required for this to work.

How the module finds the EKS cluster

This is the key part: the k8s module doesn't know about EKS at all. It just talks to a Kubernetes API endpoint, exactly like kubectl does. It finds that endpoint the same way kubectl and the Python kubernetes client always do — by reading a kubeconfig file, resolved in this order:

1. kubeconfig: param passed explicitly to the task, if you set one
2. the KUBECONFIG environment variable
3. ~/.kube/config on the machine running the play (i.e., on localhost)

So the actual EKS-awareness lives entirely in that kubeconfig file — specifically in how it was generated. For EKS, you'd normally have run:

```
aws eks update-kubeconfig --name <cluster-name> --region <region>
```

That writes an entry into ~/.kube/config with:

- the cluster's API server URL (the EKS endpoint)
- the cluster CA cert
- a user entry configured to run `aws eks get-token` (or the `aws-iam-authenticator` binary) as an exec credential plugin — this is how EKS does auth instead of a static token: every API call triggers a fresh call to the AWS CLI to mint a short-lived token from your IAM credentials.

So the full chain is:

ansible task (hosts: localhost)

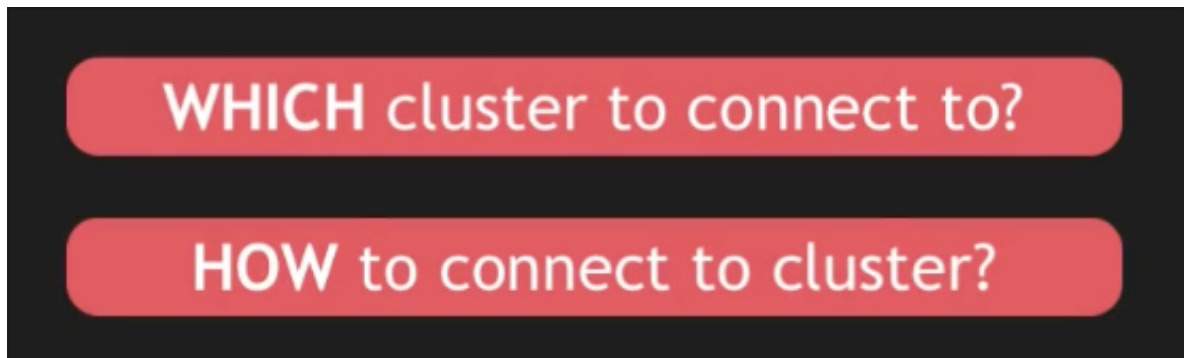
- runs k8s module as a local Python process
- module reads ~/.kube/config (or \$KUBECONFIG)
- finds EKS API server URL + exec-based auth
- shells out to ``aws eks get-token`` using your local AWS credentials/IAM role
- gets a bearer token, calls the EKS API server

One practical implication for your play: whatever machine actually runs this Ansible play (your laptop, or a CI runner, or wherever) needs:

- a valid kubeconfig pointing at the EKS cluster (or KUBECONFIG env var set)
- AWS credentials available (env vars, ~/.aws/credentials, or an instance/task IAM role) that map to an IAM identity with an entry in the cluster's aws-auth ConfigMap / EKS access entries

If you're running this from CI (Jenkins, per your other project) rather than a laptop, that's usually the part that needs the most care — making sure the CI agent has both the kubeconfig and the right IAM role, not a human's local config.

— end tangent —



Ansible needs the cluster's

- Address
- credentials

Which is exactly what we have in 'kubeconfig_myapp_eks-cluster'

```
terraform > kubeconfig_myapp-eks-cluster

providers.tf x eks-cluster.tf x kubeconfig_myapp-eks-cluster x

1 apiVersion: v1
2 clusters:
3   - cluster:
4     certificate-authority-data: LS0tLS1CRUdJ
5       server: https://69DC84BBFBE791941D1D1F3A
6       name: arn:aws:eks:eu-west-2:706976333940:c
7 contexts:
8   - context:
9       cluster: arn:aws:eks:eu-west-2:706976333
10      user: arn:aws:eks:eu-west-2:706976333940
11      name: arn:aws:eks:eu-west-2:706976333940:c
12 current-context: arn:aws:eks:eu-west-2:70697
13 kind: Config
14 preferences: {}
15 users:
```

If we check the k8s module documentation there is a 'kubeconfig' attribute

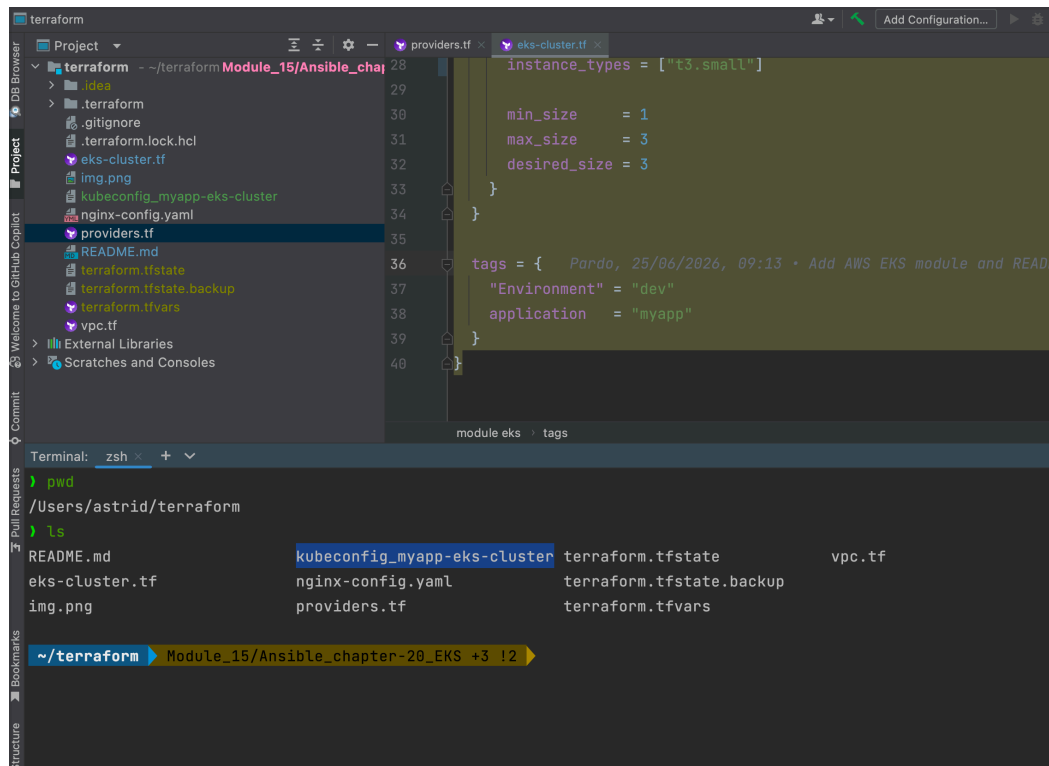
kubeconfig any	Path to an existing Kubernetes config file. If not provided, and no other connection options are provided, the Kubernetes client will attempt to load the default configuration file from <code>~/.kube/config</code> . Can also be specified via <code>K8S_AUTH_KUBECONFIG</code> environment variable. Multiple Kubernetes config file can be provided using separator <code>;</code> for Windows platform or <code>:</code> for others platforms. The kubernetes configuration can be provided as dictionary. This feature requires a python kubernetes client version <code>>= 17.17.0</code> . Added in version 2.2.0.
-------------------	--

Which points to the path of the kubeconfig file.

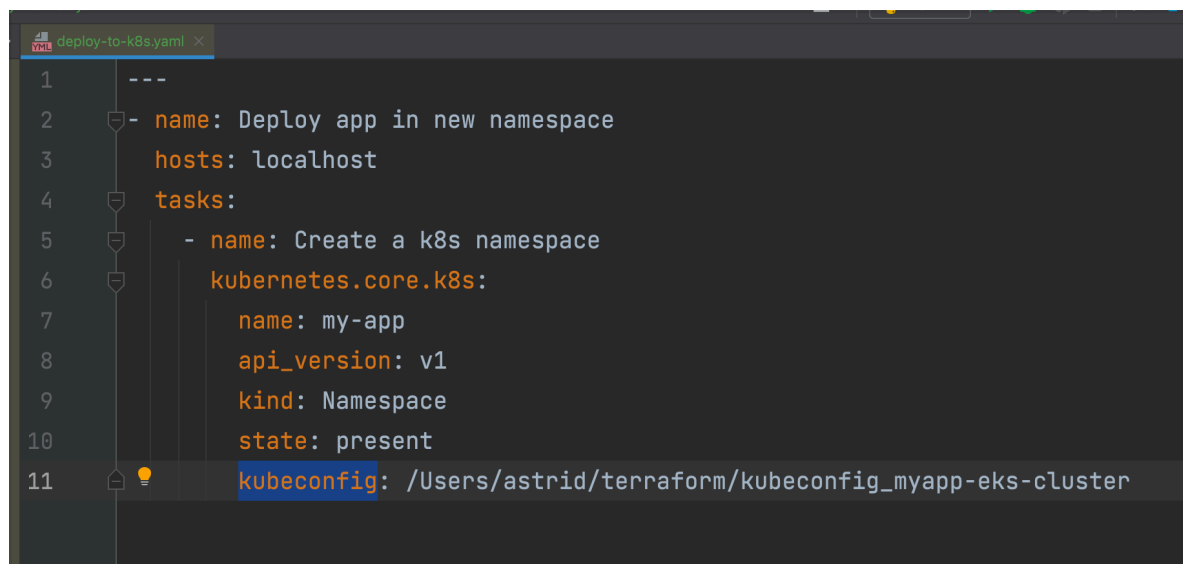
If we don't specify the path by default will use `~/.kube/config`

Load the default configuration file from `~/.kube/config`.

So I get the path



Our kubeconfig is not the default so we need to set the kubeconfig attribute to the path '/Users/astrid/terraform/kubeconfig_myapp-eks-cluster'



So now ansible is configured to read the kubeconfig context information from 'kubeconfig_myapp-eks-cluster' and get the cluster location and the certificates to authenticate.

And the next configuration has to do with the requirements for the k8s module

Requirements

The below requirements are needed on the host that executes this module.

- python >= 3.9
- kubernetes >= 24.2.0
- PyYAML >= 3.11
- jsonpatch

So the hosts that executes the module in this case my laptop (localhost) needs to have the above.

I need to check if the above python modules are installed.

Here are commands to check each of those on the machine that will run the module (your localhost in this case):

```
# python >= 3.9
python3 --version
```

```
# which python Ansible itself will use (important if you have multiple pythons/
venvs)
ansible -m setup -a "filter=ansible_python_version" localhost
```

```
# kubernetes python package >= 24.2.0
python3 -c "import kubernetes; print(kubernetes.__version__)"
# or
pip3 show kubernetes
```

```
# PyYAML >= 3.11
python3 -c "import yaml; print(yaml.__version__)"
# or
pip3 show pyyaml
```

```
# jsonpatch (any version, just needs to be installed)
python3 -c "import jsonpatch; print(jsonpatch.__version__)"
# or
pip3 show jsonpatch
```

One-liner to check all four at once:

```
python3 -c "
import sys, kubernetes, yaml, jsonpatch
```

```

print('python:', sys.version)
print('kubernetes:', kubernetes.__version__)
print('PyYAML:', yaml.__version__)
print('jsonpatch:', jsonpatch.__version__)
"

```

Important gotcha: `python3 --version` in your terminal checks your shell's Python, but Ansible may run tasks under a different interpreter (e.g. inside a venv, or a different `ansible_python_interpreter`). Since your play is hosts: localhost, confirm which interpreter Ansible is actually using:

```
ansible localhost -m debug -a "var=ansible_playbook_python"
```

```

>> python3 --version
Python 3.14.6
>> ansible -m setup -a "filter=ansible_python_version" localhost
[WARNING]: No inventory was parsed, only implicit localhost is available
localhost | SUCCESS => {
  "ansible_facts": {
    "ansible_python_version": "3.14.6"
  },
  "changed": false
}
>> python3 -c "import kubernetes; print(kubernetes.__version__)"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import kubernetes; print(kubernetes.__version__)
    ~~~~~
ModuleNotFoundError: No module named 'kubernetes'
>> python3 -c "import yaml; print(yaml.__version__)"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import yaml; print(yaml.__version__)
    ~~~~~
ModuleNotFoundError: No module named 'yaml'
>> python3 -c "import jsonpatch; print(jsonpatch.__version__)"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import jsonpatch; print(jsonpatch.__version__)
    ~~~~~
ModuleNotFoundError: No module named 'jsonpatch'

```

`ansible_python_version: 3.14.6` matching your `python3 --version` output confirms Ansible is using the same interpreter as your shell. That satisfies the `python >= 3.9` requirement.

We need to install 3 modules.

pip install



- ▶ Pip defaults to installing Python packages to a system directory (such as `/usr/local/libpython3.7`).
 - This **requires root access**
- ▶ `--user` makes pip install packages in your home directory instead
 - Requires **no special privileges**

— tangent pip3 install pyyaml --user vs pip3 install pyyaml —

Q:

`pip3 install pyyaml --user` VS `pip3 install pyyaml`

A:

`--user` changes where the package gets installed, not what gets installed.

pip3 install pyyaml (no flag)

Installs into the interpreter's main site-packages — typically requires write access to that location. On a system Python (not a venv), this often needs `sudo` or fails with a permissions error. On modern Python (3.11+), if the system Python is "externally managed" (PEP 668 — common on macOS Homebrew, Debian/Ubuntu system Python), this will often refuse outright with an externally-managed-environment error.

pip3 install pyyaml --user

Installs into your user site-packages instead (`~/Library/Python/3.x/lib/python/site-packages` on macOS, or `~/.local/lib/pythonX.Y/site-packages` on Linux). No `admin/sudo` needed, and it won't touch or conflict with system-managed packages. This is generally the safer default when installing directly onto your system Python (as opposed to inside a venv).

A few practical notes for your case:

- If you're inside a virtualenv, `--user` is irrelevant/ignored — venvs already isolate you, so plain `pip3 install pyyaml` is correct there.
- Since Ansible resolved to your plain python3 (3.14.6, not a venv) earlier, `--user` is likely the right choice to avoid permission errors or the externally-managed-environment block.
- One gotcha: Ansible needs to find the package at runtime using the same interpreter it's using to run modules. `--user` installs are still importable by that

same interpreter (they go on sys.path automatically), so this works fine — just don't accidentally install with a different python3/pip3 (e.g. a pyenv or Homebrew one) than the one ansible_playbook_python actually resolves to, or Ansible won't see it.

If pip3 install pyyaml fails with externally-managed-environment, try --user first before reaching for --break-system-packages.

— end tangent —

-> pip3 install pyyaml --user

```
> pip3 install pyyaml --user
error: externally-managed-environment

× This environment is externally managed
➤ To install Python packages system-wide, try brew install
  xyz, where xyz is the package you are trying to
  install.

If you wish to install a Python library that isn't in Homebrew,
use a virtual environment:

python3 -m venv path/to/venv
source path/to/venv/bin/activate
python3 -m pip install xyz

If you wish to install a Python application that isn't in Homebrew,
it may be easiest to use 'pipx install xyz', which will manage a
virtual environment for you. You can install pipx with

brew install pipx

You may restore the old behavior of pip by passing
the '--break-system-packages' flag to pip, or by adding
'break-system-packages = true' to your pip.conf file. The latter
will permanently disable this error.

If you disable this error, we STRONGLY recommend that you additionally
pass the '--user' flag to pip, or set 'user = true' in your pip.conf
file. Failure to do this can result in a broken Homebrew installation.

Read more about this behavior here: <https://peps.python.org/pep-0668/>

note: If you believe this is a mistake, please contact your Python installation or OS dis
on or OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.
```

👉 this confirms you're on Homebrew's Python, which blocks both plain and --user installs since PEP 668. You have two real options:

```

Option 1 (recommended): virtual environment

python3 -m venv ~/.venvs/ansible-eks
source ~/.venvs/ansible-eks/bin/activate
pip install "kubernetes>=24.2.0" "PyYAML>=3.11" jsonpatch ansible

The catch for you specifically: Ansible must run its modules using this venv's Python, not the Homebrew one it found earlier (3.14.6). Point it there either per-play or globally.

In your playbook:
- name: Deploy app in new namespace
  hosts: localhost
  vars:
    ansible_python_interpreter: "{{ '~/.venvs/ansible-eks/bin/python3' | expanduser }}"
  tasks:
    ...

Or in ansible.cfg:
[defaults]
interpreter_python = ~/.venvs/ansible-eks/bin/python3

Verify it picked up correctly:
ansible localhost -m debug -a "var=ansible_playbook_python"
# should now show the venv path, not /opt/homebrew/.../python3

```

```

Option 2 (quick, riskier): override the guard

pip3 install "kubernetes>=24.2.0" "PyYAML>=3.11" jsonpatch --break-system-packages --user

This works but mixes third-party packages into Homebrew's managed Python install – the exact scenario PEP 668 exists to prevent, and it can break brew-installed tools that depend on a clean system Python later.

For a portfolio/demo project like this I'd go with the venv – it's the standard, "correct" way to run Ansible with Python collections anyway, and avoids polluting your Homebrew install.

```

So I will do option 1 from Pycharm terminal as my ansible project already has a venv.

I need to check that venv is activated
-> which python3

```

> which python3
/usr/local/bin/python3

```

That's not the venv — that's your system Python. It's not auto-activated in this terminal session. Activate it manually:

-> source .venv/bin/activate

Then re-check:

-> which python3

```

> source .venv/bin/activate
> which python3
/Users/astrid/PythonProject/Ansible/.venv/bin/python3

~/Pyt/Ansible ▶ Module_15/Ans...r-20_Ansible *1 +3 !2 ?1

```

now points to ~/PythonProject/Ansible/.venv/bin/python3.

Then install:

```
-> pip install "kubernetes>=24.2.0" "PyYAML>=3.11" jsonpatch
```

— tangent pip vs pip3 inside a venv —

Q:

why pip and not pip3?

A:

Once the venv is activated, it doesn't matter — pip and pip3 are both symlinks pointing to the same thing inside `.venv/bin/`:

```
ls -la .venv/bin/pip*
```

```
# pip -> pip3 -> pip3.x (all same file, just symlinked)
```

A venv only ever contains one Python version, so there's no ambiguity between Python 2 pip and Python 3 pip the way there sometimes is on a system install. Inside an activated venv, pip, pip3, and pip3.13 (or whatever your version is) all resolve identically.

I used pip3 earlier because at that point you were not inside the venv — you were on system Python, where pip might not exist at all, or (on older systems) could point to Python 2. Being explicit with pip3 avoided ambiguity there.

Now that you've activated `.venv`, either form is fine — I'd just use pip for brevity since the venv makes it unambiguous.

— end tangent —

```
> pip install "kubernetes>=24.2.0" "PyYAML>=3.11" jsonpatch
Collecting kubernetes>=24.2.0
  Downloading kubernetes-36.0.3-py2.py3-none-any.whl.metadata (1.8 kB)
Collecting PyYAML>=3.11
```

— tangent quotation marks missing for installing jsonpatch —

The quotes matter because of the `>=` inside them. In your shell (zsh/bash), `>` is a redirection operator — unquoted, `kubernetes>=24.2.0` would be parsed as "run command `kubernetes`, redirect output to a file named `=24.2.0`". Quoting protects the `>=` from the shell so it gets passed to pip literally as a version specifier.

jsonpatch has no version constraint — I only said "any version, just needs to be installed" — so there's no `>=`, no special characters, nothing for the shell to misinterpret. Quotes are harmless there but unnecessary.

If you want to pin a minimum version for jsonpatch too, quote it the same way:

```
pip3 install "kubernetes>=24.2.0" "PyYAML>=3.11" "jsonpatch>=1.0"
```

—— End tangent ——

I can check the installation was succesfull:

```
-> python3 -c "  
import kubernetes, yaml, jsonpatch  
print('kubernetes:', kubernetes.__version__)  
print('PyYAML:', yaml.__version__)  
print('jsonpatch:', jsonpatch.__version__)  
"
```

Or via pip:

```
-> pip show kubernetes PyYAML jsonpatch
```

And they are installed in the venv

```
> python3 -c "  
dquote> import kubernetes, yaml, jsonpatch  
dquote> print('kubernetes:', kubernetes.__version__)  
dquote> print('PyYAML:', yaml.__version__)  
dquote> print('jsonpatch:', jsonpatch.__version__)  
dquote> "  
kubernetes: 36.0.3  
PyYAML: 6.0.3  
jsonpatch: 1.33
```

Next, in my case Ansible must run its modules using this venv's Python, not the Homebrew one it found earlier (3.14.6). Point it there either per-play or globally with the ansible.cfg

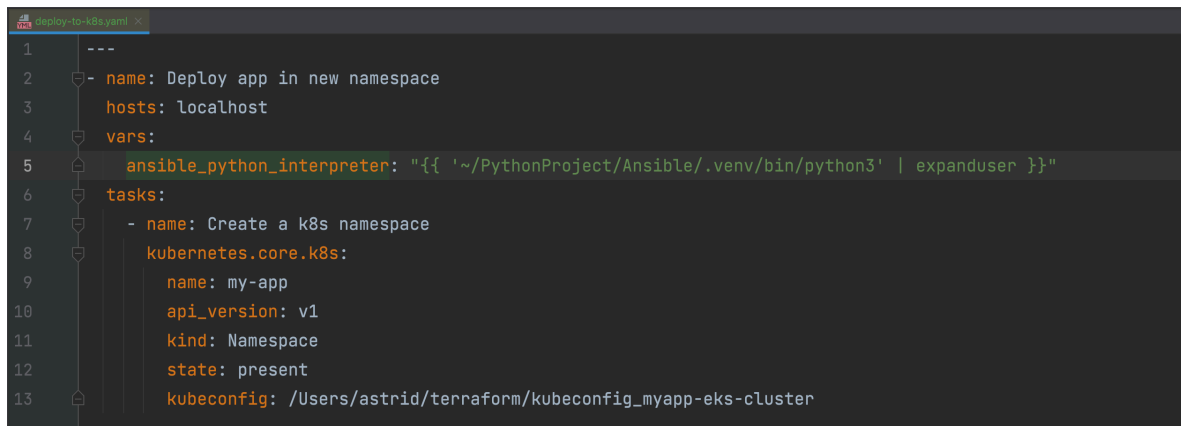
```

In your playbook:
- name: Deploy app in new namespace
  hosts: localhost
  vars:
    ansible_python_interpreter: "{{ '~/.venvs/ansible-eks/bin/python3' | expanduser }}"
  tasks:
    ...

Or in ansible.cfg:
[defaults]
interpreter_python = ~/.venvs/ansible-eks/bin/python3

```

If I do it in the play:

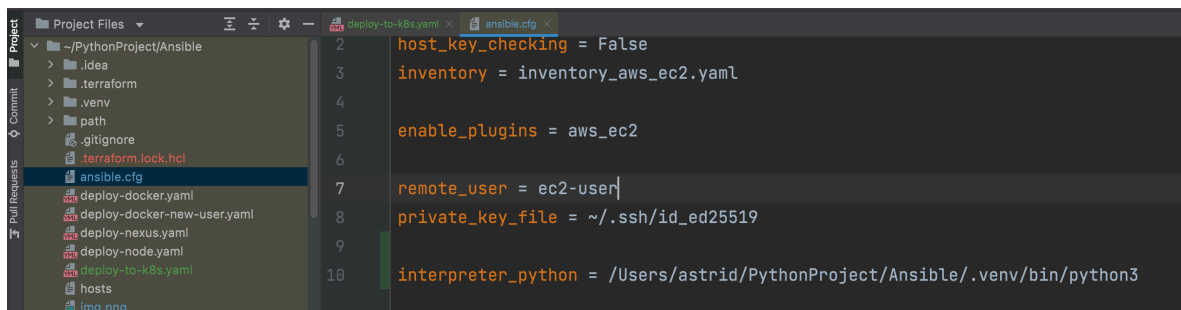


```

1  ---
2  - name: Deploy app in new namespace
3    hosts: localhost
4    vars:
5      ansible_python_interpreter: "{{ '~/.PythonProject/Ansible/.venv/bin/python3' | expanduser }}"
6    tasks:
7      - name: Create a k8s namespace
8        kubernetes.core.k8s:
9          name: my-app
10         api_version: v1
11         kind: Namespace
12         state: present
13         kubeconfig: /Users/astrid/terraform/kubeconfig_myapp-eks-cluster

```

But I prefer globally:



```

2  host_key_checking = False
3  inventory = inventory_aws_ec2.yaml
4
5  enable_plugins = aws_ec2
6
7  remote_user = ec2-user
8  private_key_file = ~/.ssh/id_ed25519
9
10 interpreter_python = /Users/astrid/PythonProject/Ansible/.venv/bin/python3

```

And I need to edit the ansible.cfg to use 'hosts' file as inventory because 'hosts' doesn't have 'localhost'

```
deploy-to-k8s.yaml × hosts × ansible.cfg ×
[nexus_server]
188.166.148.119 ansible_ssh_p

[docker_server]
35.178.50.115 ansible_ssh_pri
```

Ansible falls back to its implicit built-in localhost (with connection: local), same as if you had no inventory at all.

If there was an entry 'localhost' in 'hosts' file then that entry's config applies instead of the implicit default. Hence the 'hosts' file doesn't affect the play at all.

Let's execute ansible playbook

-> ansible-playbook deploy-to-k8s.yaml

```
> ansible-playbook deploy-to-k8s.yaml

PLAY [Deploy app in new namespace] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [Create a k8s namespace] *****
changed: [localhost]

PLAY RECAP *****
localhost : ok=2 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

Now let's check the cluster to see if the namespace was created.

- Set the KUBECONFIG env var to the terraform kubeconfig path
-> export KUBECONFIG=/Users/astrid/terraform/kubeconfig_myapp-eks-cluster

- Now get the namespaces of the eks cluster
-> kubectl get ns

```
> export KUBECONFIG=/Users/astrid/terraform/kubeconfig_myapp-eks-cluster
> kubectl get ns
```

NAME	STATUS	AGE
default	Active	4h16m
kube-node-lease	Active	4h16m
kube-public	Active	4h16m
kube-system	Active	4h16m
my-app	Active	3m3s

And we can see that the namespace 'my-app' is created 🥳

Now let's deploy an application from k8s configuration file using k8s ansible module.

Deploy app in new namespace

We will deploy nginx app.

This time we don't want to copy the configuration of the component. Like we did in lines 7-9.

This time we want to use the k8s file itself (instead of copying its contents and paste them into the play)

Nan has a Nginx-config.yaml ready

```
% cat ~/demo-projects/K8s-Files/nginx-config.yaml
```

```
---
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: nginx-deployment
```

```
spec:
```

```
  selector:
```

```
    matchLabels:
```

```
      app: nginx
```

```
  replicas: 1
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: nginx
```

```
  spec:
```

```
    containers:
```

```
      - name: nginx
```

```
        image: nginx:
```

```
        ports:
```

```
          - containerPort: 80
```

```
---
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: nginx
```

```
  labels:
```

```
    app: nginx
```

```
spec:
```

```
  ports:
```

```
    - name: http
```

```
      port: 80
```

I found a similar file in my terraform project

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: nginx
5 spec:
6   selector:
7     matchLabels:
8       app: nginx
9   replicas: 1
10  template:
11    metadata:
12      labels:
13        app: nginx
14    spec:
15      containers:
16      - name: nginx
17        image: nginx
18        ports:
19        - containerPort: 80
20 ---
21 apiVersion: v1
22 kind: Service
23 metadata:
```

So we want to deploy using the file above and for that we will use the attribute 'src' in the ansible k8s module

src path	Provide a path to a file containing a valid YAML definition of an object or objects to be created or updated. Mutually exclusive with <code>resource_definition</code>. NOTE: <code>kind</code>, <code>api_version</code>, <code>name</code>, and <code>namespace</code> will be overwritten by corresponding values found in the configuration read in from the <code>src</code> file. Reads from the local file system. To read from the Ansible controller's file system, including vaulted files, use the file lookup plugin or template lookup plugin, combined with the <code>from_yaml</code> filter, and pass the result to <code>resource_definition</code>. See Examples below. The URL to manifest files that can be used to create the resource. Added in version 2.4.0. Mutually exclusive with <code>template</code> in case of <code>kubernetes.core.k8s</code> module.
--------------------	--

'src' will point at the file that contains k8s yaml configuration

```
1 ---
2 - name: Deploy app in new namespace
3   hosts: localhost
4   tasks:
5     - name: Create a k8s namespace
6       kubernetes.core.k8s:
7         name: my-app
8         api_version: v1
9         kind: Namespace
10        state: present
11        kubeconfig: /Users/astrid/terraform/kubeconfig_myapp-eks-cluster
12    - name: Deploy nginx app
13      kubernetes:
14        src: /Users/astrid/terraform/nginx-config.yaml
```

And we also need to pass kubeconfig context to the new task

```
1 ---
2 - name: Deploy app in new namespace
3   hosts: localhost
4   tasks:
5     - name: Create a k8s namespace
6       kubernetes.core.k8s:
7         name: my-app
8         api_version: v1
9         kind: Namespace
10        state: present
11        kubeconfig: /Users/astrid/terraform/kubeconfig_myapp-eks-cluster
12    - name: Deploy nginx app
13      kubernetes:
14        src: /Users/astrid/terraform/nginx-config.yaml
15        kubeconfig: /Users/astrid/terraform/kubeconfig_myapp-eks-cluster
```

If the k8s config file doesn't specify a namespace then the deployment and service will be created in the default namespace. Our file doesn't have a namespace specified in the metadata, so we can override that in ansible.

So we will do that, will use the new namespace 'my-aap' and we will create the deployment and the service there.

```
1 ---
2 - name: Deploy app in new namespace
3   hosts: localhost
4   tasks:
5     - name: Create a k8s namespace
6       kubernetes.core.k8s:
7         name: my-app
8         api_version: v1
9         kind: Namespace
10        state: present
11        kubeconfig: /Users/astrid/terraform/kubeconfig_myapp-eks-cluster
12    - name: Deploy nginx app
13      kubernetes:
14        src: /Users/astrid/terraform/nginx-config.yaml
15        kubeconfig: /Users/astrid/terraform/kubeconfig_myapp-eks-cluster
16        namespace: my-app
```

And let's execute the playbook

-> `ansible-playbook deploy-to-k8s.yaml`

```

> ansible-playbook deploy-to-k8s.yaml
[ERROR]: couldn't resolve module/action 'kubernetes'. This often indicates a misspelling, missing collection, or incorrect module path.
Origin: /Users/astrid/PythonProject/Ansible/deploy-to-k8s.yaml:12:7

10     state: present
11     kubeconfig: /Users/astrid/terraform/kubeconfig_myapp-eks-cluster
12 - name: Deploy nginx app
    ^ column 7

```

It failed, because the module's name was wrong, so I corrected it:

And execute the playbook again:

-> ansible-playbook deploy-to-k8s.yaml

```

12 - name: Deploy nginx app
13   kubernetes.core.k8s:
14     src: /Users/astrid/terraform/nginx-config.yaml
15     kubeconfig: /Users/astrid/terraform/kubeconfig_myapp-eks-cluster
16     namespace: my-app

```

```

> ansible-playbook deploy-to-k8s.yaml

PLAY [Deploy app in new namespace] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [Create a k8s namespace] *****
ok: [localhost]

TASK [Deploy nginx app] *****
changed: [localhost]

PLAY RECAP *****
localhost                : ok=3    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

```

Now let's check the EKS cluster

-> kubectl get pod -n my-app

```

> kubectl get pod -n my-app

```

NAME	READY	STATUS	RESTARTS	AGE
nginx-5cf8dc6bc5-s6l6x	1/1	Running	0	2m56s

We have the pod!

-> kubectl get svc -n my-app

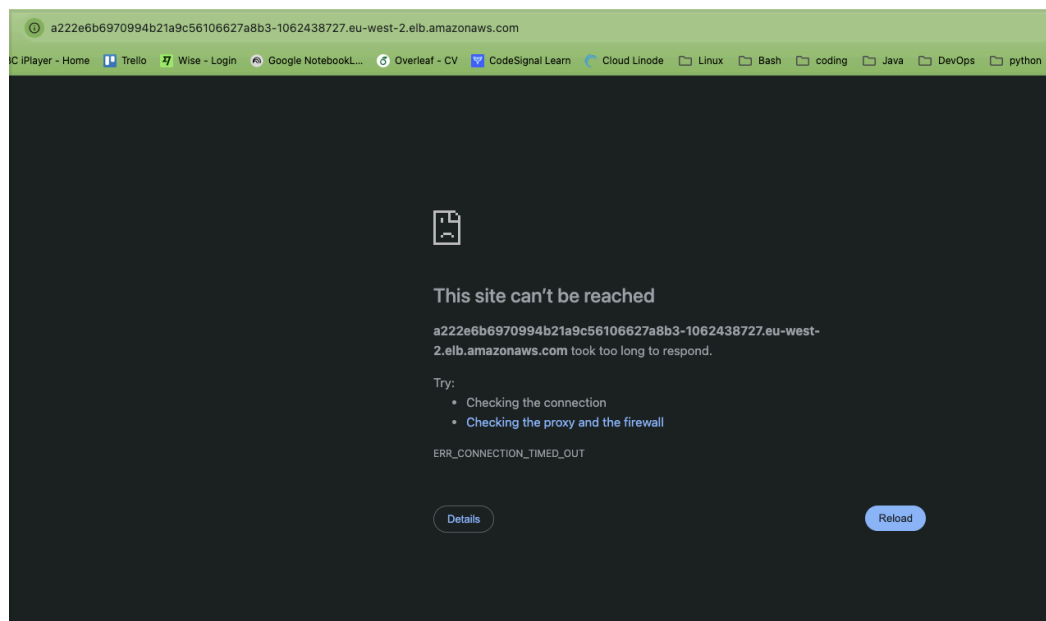
```
kubectl get svc -n my-app
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
nginx	LoadBalancer	172.20.75.112	a222e6b6970994b21a9c56106627a8b3-1062438727.eu-west-2.elb.amazonaws.com	80:30123/TCP

3m45s

And we have the service of type loadbalancer as well 🥳

And we got the url of the load balancer so we can check in the browser:



```
-> aws elb describe-load-balancers --query "LoadBalancerDescriptions[?contains(DNSName, 'a222e6b6970994b21a9c56106627a8b3')].LoadBalancerName" --output text
```

And it returns: a222e6b6970994b21a9c56106627a8b3

One node is working and the other two are out of service.

```
-> aws elb describe-instance-health --load-balancer-name a222e6b6970994b21a9c56106627a8b3
```

matches your ELB's DNS name

(a222e6b6970994b21a9c56106627a8b3-...elb.amazonaws.com) to get its LoadBalancerName.

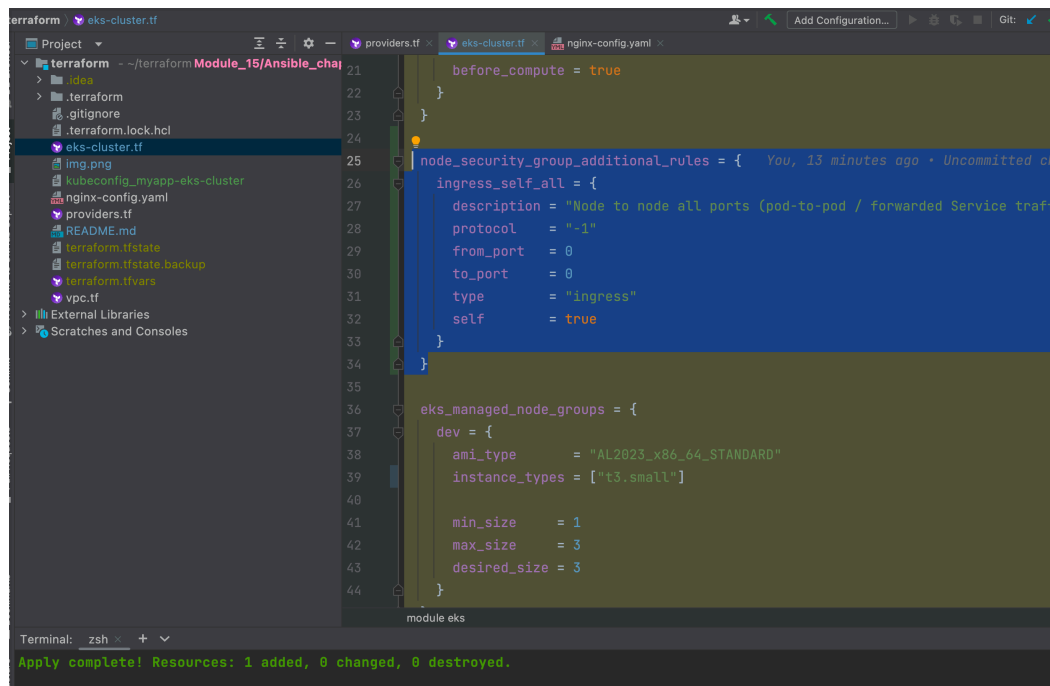
AI says the issue is:

this is actually the official terraform-aws-modules/eks module's intentional default, not a mistake unique to your config. The module deliberately ships only narrow default rules (control-plane webhooks, the ephemeral-port range, DNS) and expects you to add any additional ports your workloads need via `node_security_group_additional_rules`. Since this is a dev cluster you'll likely deploy various apps to (each on different ports), the cleanest fix is a self-

referencing "allow all traffic between nodes" rule — this is what most people add for exactly this reason.

I edited the eks-cluster.tf terraform file by adding the code below

```
• Update(terraform/eks-cluster.tf)
└─ User rejected update to terraform/eks-cluster.tf
  22   }
  23   }
  24
  25 + node_security_group_additional_rules = {
  26 +   ingress_self_all = {
  27 +     description = "Node to node all ports (pod-to-pod / forwarded Service traffic)"
  28 +     protocol    = "-1"
  29 +     from_port   = 0
  30 +     to_port     = 0
  31 +     type        = "ingress"
  32 +     self        = true
  33 +   }
  34 + }
  35 +
  36 eks_managed_node_groups = {
  37   dev = {
  38     ami_type = "AL2023_x86_64_STANDARD"
```



-> terraform apply

No need to re run the playbook — the fix was purely at the AWS networking layer (a security group rule), not the Kubernetes objects themselves. Your Deployment and Service are already applied and correctly configured; nothing about them needs to change.

And run

-> aws elb describe-instance-health --load-balancer-name

a222e6b6970994b21a9c56106627a8b3

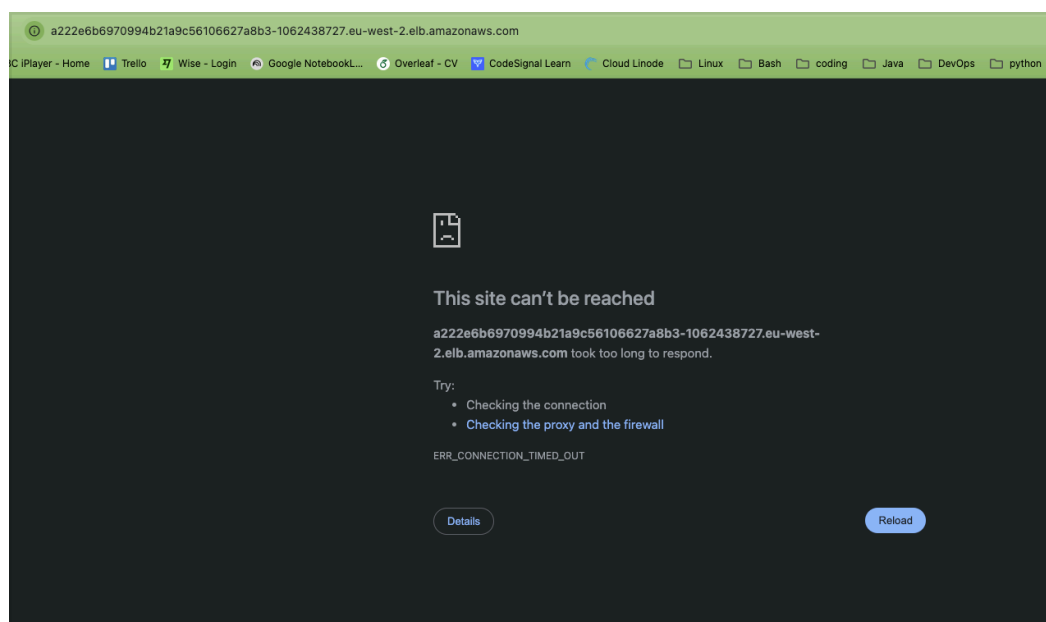
and now all 3 nodes are working

```

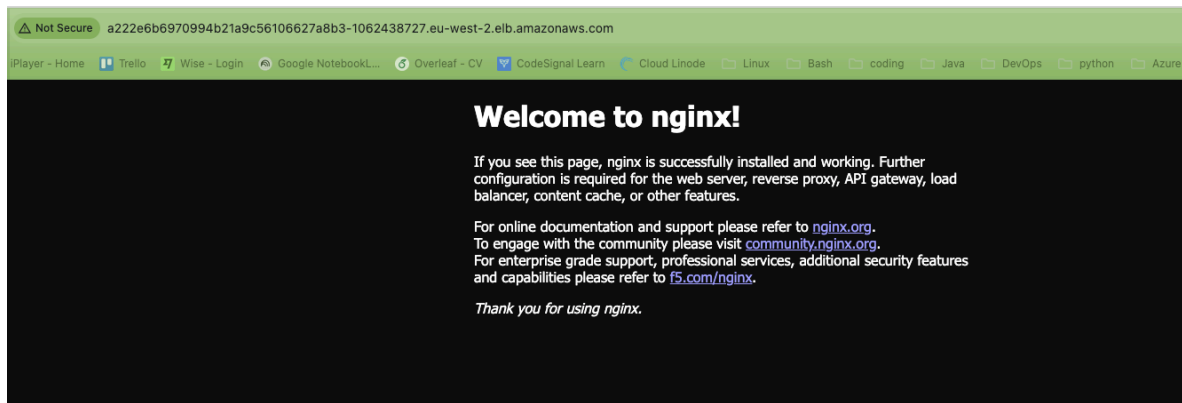
{
  "InstanceStates": [
    {
      "InstanceId": "i-078fb343639d77419",
      "State": "InService",
      "ReasonCode": "N/A",
      "Description": "N/A"
    },
    {
      "InstanceId": "i-0929bf4e6145b2035",
      "State": "InService",
      "ReasonCode": "N/A",
      "Description": "N/A"
    },
    {
      "InstanceId": "i-0fe979de41abfd128",
      "State": "InService",
      "ReasonCode": "N/A",
      "Description": "N/A"
    }
  ]
}

```

And it fails again in the browser



This time because it is using https instead of http, so I edit it to 'http' and it works



If we want to deploy more components we can use k8s ansible module.

If we want to write the configuration directly in ansible instead of using 'src' and point at a yaml file we can do it using 'definition' attribute

```
- name: Create a Service object from an inline definition
  kubernetes.core.k8s:
    state: present
    definition:
      apiVersion: v1
      kind: Service
      metadata:
        name: web
        namespace: testing
        labels:
          app: galaxy
          service: web
      spec:
        selector:
          app: galaxy
          service: web
        ports:
          - protocol: TCP
            targetPort: 8000
            name: port-8000-tcp
            port: 8000
```

Set environment variable for kubeconfig

If we have many plays in the playbook it will be inconvenient to repeat the attribute 'kubeconfig'

```
tasks:
  - name: Create a k8s namespace
    kubernetes.core.k8s:
      name: my-app
      api_version: v1
      kind: Namespace
      state: present
      kubeconfig: /Users/nana/terraform/kubeconfig_myapp-eks-cluster
  - name: Deploy nginx app
    kubernetes.core.k8s:
      src: ~/demo-projects/K8s-Files/nginx-config.yaml
      state: present
      kubeconfig: /Users/nana/terraform/kubeconfig_myapp-eks-cluster
      namespace: my-app
```

There is a better way to define the 'kubeconfig' attribute and make it reachable by the whole playbook.

The documentation says that we can set an env var
-> K8S_AUTH_KUBECONFIG

kubeconfig any	Path to an existing Kubernetes config file. If not provided, and no other connection options are provided, the Kubernetes client will use the default configuration. Can also be specified via <code>K8S_AUTH_KUBECONFIG</code> environment variable. Multiple Kubernetes config file can be provided using separator ';' for Windows platform or ':' for others platforms. The kubernetes configuration can be provided as dictionary. This feature requires a python kubernetes client version >= 6.0.0
--------------------------	--

And ansible will use it to get the kubeconfig anytime it needs it
-> export K8S_AUTH_KUBECONFIG=/Users/astrid/terraform/
kubeconfig_myapp-eks-cluster

Now we can remove the attribute from the playbook

```
Users > astrid > PythonProject > Ansible > deploy-to-k8s.yaml
1 ---
2 - name: Deploy app in new namespace
3   hosts: localhost
4   tasks:
5     - name: Create a k8s namespace
6       kubernetes.core.k8s:
7         name: my-app
8         api_version: v1
9         kind: Namespace
10        state: present
11     - name: Deploy nginx app
12       kubernetes.core.k8s:
13         src: /Users/astrid/terraform/nginx-config.yaml
14         namespace: my-app

Document 1/1 > Item 1/1 > tasks: > Item 1/2 > kubernetes.core.k8s: > state: > present
Terminal: Local > + > -
aws: [ERROR]: An error occurred (TargetNotConnected) when calling the StartSession operation.
> clear
> export K8S_AUTH_KUBECONFIG=/Users/astrid/terraform/kubeconfig_myapp-eks-cluster
~/Pyt/Ansible > Module_15/Ansible-20_Ansible *1 +3 !3 ?1
```

I missed 'state: present' between lines 13 and 14 🙅

```
11     - name: Deploy nginx app
12       kubernetes.core.k8s:
13         src: /Users/astrid/terraform/nginx-config.yaml
14         state: present
15         namespace: my-app
```

And we execute the playbook again
-> ansible-playbook deploy-to-k8s.yaml

```
PLAY [Deploy app in new namespace] *****

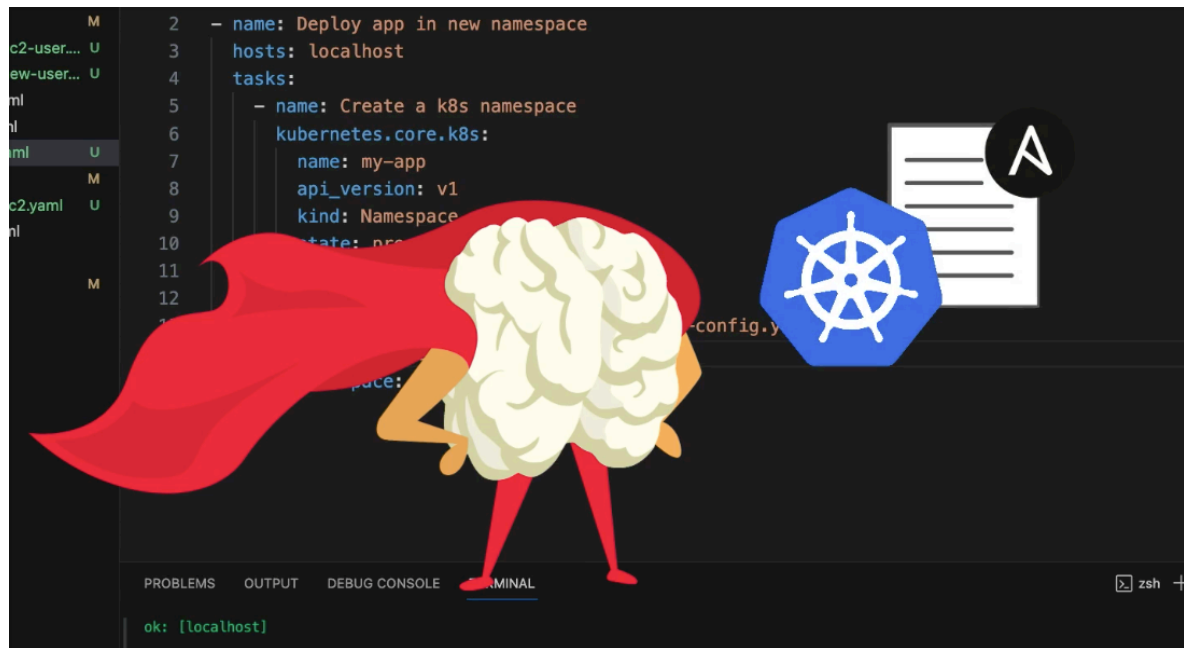
TASK [Gathering Facts] *****
ok: [localhost]

TASK [Create a k8s namespace] *****
ok: [localhost]

TASK [Deploy nginx app] *****
ok: [localhost]

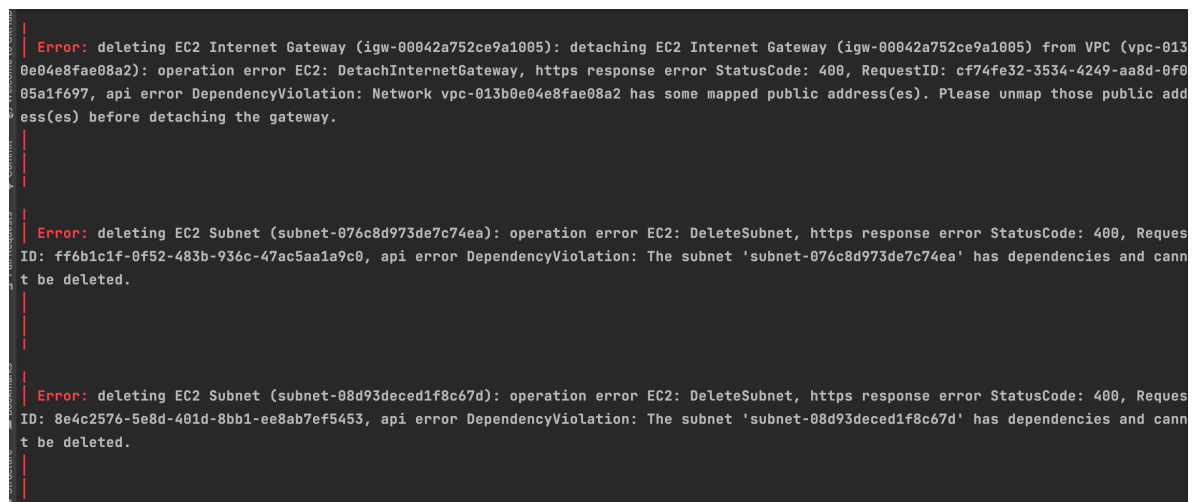
PLAY RECAP *****
localhost : ok=3  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```

And the playbook still works!



Now I destroy the infra
-> terraform destroy

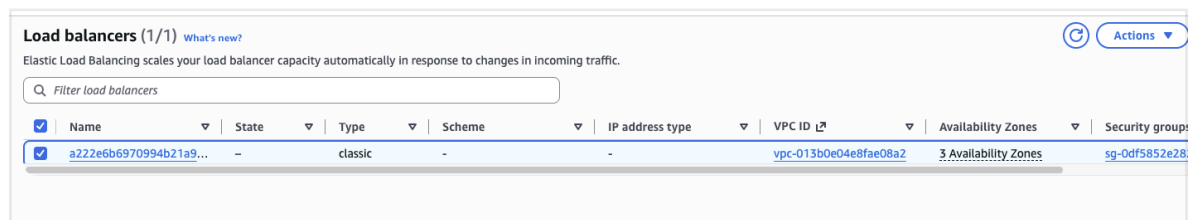
And I got some errors:



issue: This is a classic Terraform + EKS gotcha: your nginx Service's classic ELB was created by Kubernetes (via the in-tree AWS cloud provider), not by Terraform — so Terraform has no record of it and doesn't know to delete it first. Its ENIs are still sitting in those three subnets, blocking their deletion.

the leftover classic ELB (a222e6b6970994b21a9c56106627a8b3, the one from the nginx Service) is the only thing blocking cleanup. Since you've already run terraform destroy and are tearing this environment down, deleting it manually is the right next step — but flagging since it's a real delete: this permanently

removes the load balancer.



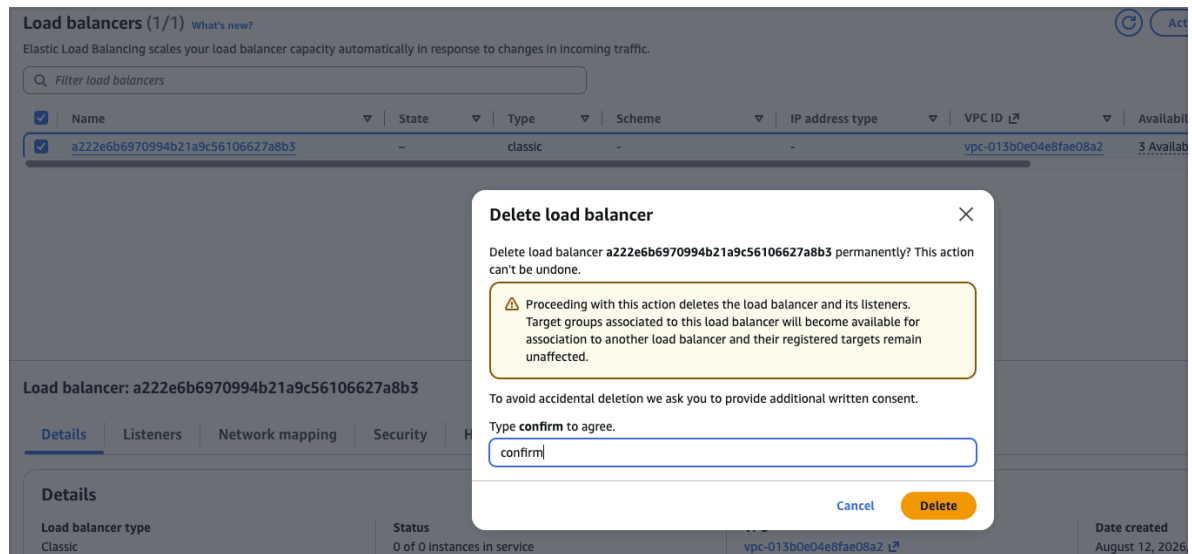
Load balancers (1/1) [What's new?](#)

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

☒	Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones	Security group
☒	a222e6b6970994b21a9c56106627a8b3	-	classic	-	-	vpc-013b0e04e8fae08a2	3 Availability Zones	sg-0df5852e28

So I delete the loadbalancer



Load balancers (1/1) [What's new?](#)

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

☒	Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones	Security group
☒	a222e6b6970994b21a9c56106627a8b3	-	classic	-	-	vpc-013b0e04e8fae08a2	3 Availab	

Delete load balancer

Delete load balancer **a222e6b6970994b21a9c56106627a8b3** permanently? This action can't be undone.

Warning: Proceeding with this action deletes the load balancer and its listeners. Target groups associated to this load balancer will become available for association to another load balancer and their registered targets remain unaffected.

To avoid accidental deletion we ask you to provide additional written consent.

Type **confirm** to agree.

confirm

[Cancel](#) [Delete](#)

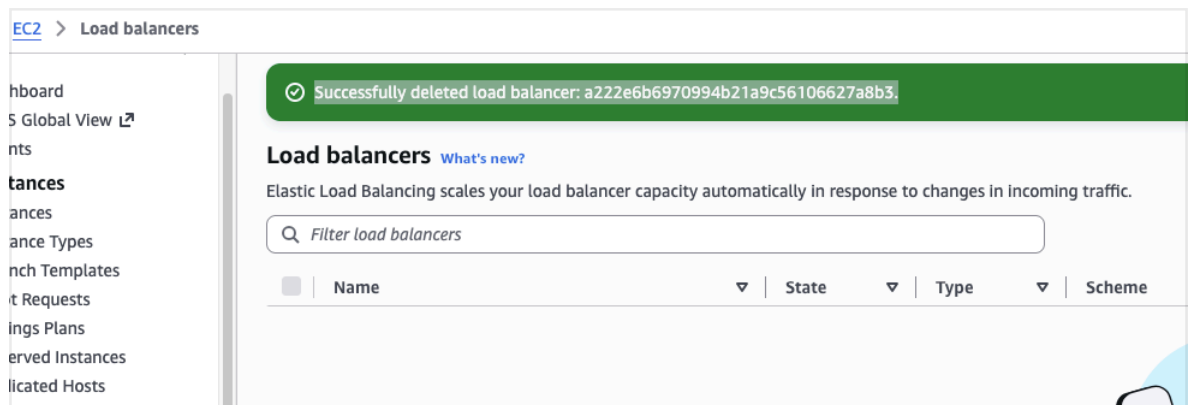
Load balancer: a222e6b6970994b21a9c56106627a8b3

[Details](#) [Listeners](#) [Network mapping](#) [Security](#)

Details

Load balancer type	Status	VPC ID	Date created
Classic	0 of 0 instances in service	vpc-013b0e04e8fae08a2	August 12, 2026

Deleted:



[EC2](#) > **Load balancers**

Successfully deleted load balancer: a222e6b6970994b21a9c56106627a8b3.

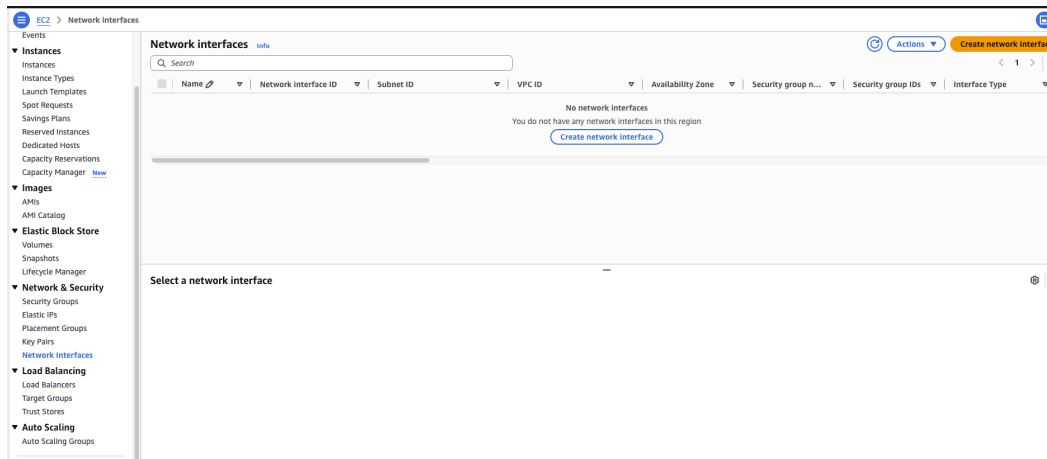
Load balancers [What's new?](#)

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

☐	Name	State	Type	Scheme
--------------------------	------	-------	------	--------

I check the ENIs and they are gone as well



-> terraform destroy

```
Plan: 0 to add, 0 to change, 5 to destroy.
```

Do you really want to destroy all resources?

Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

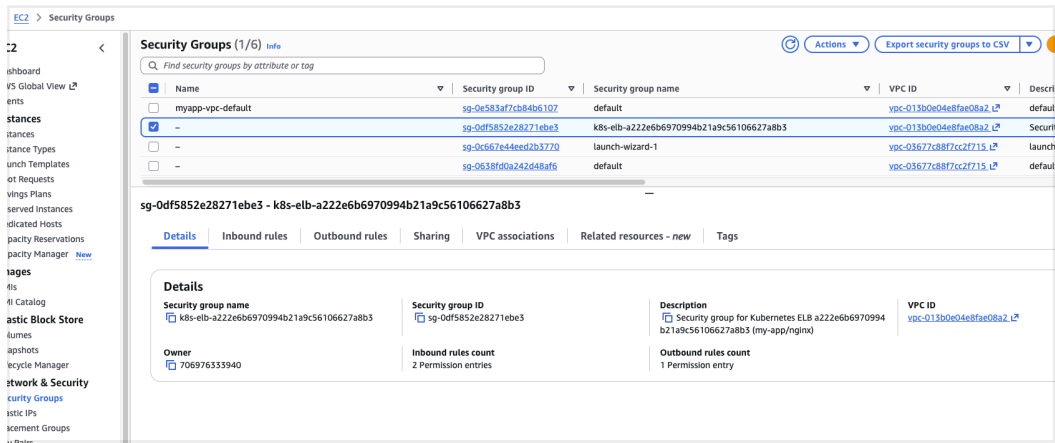
```
Enter a value: yes
```

I got one error:

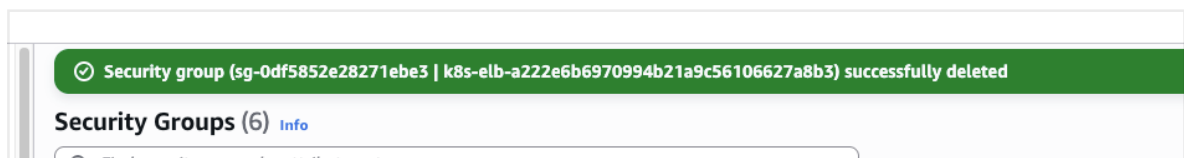
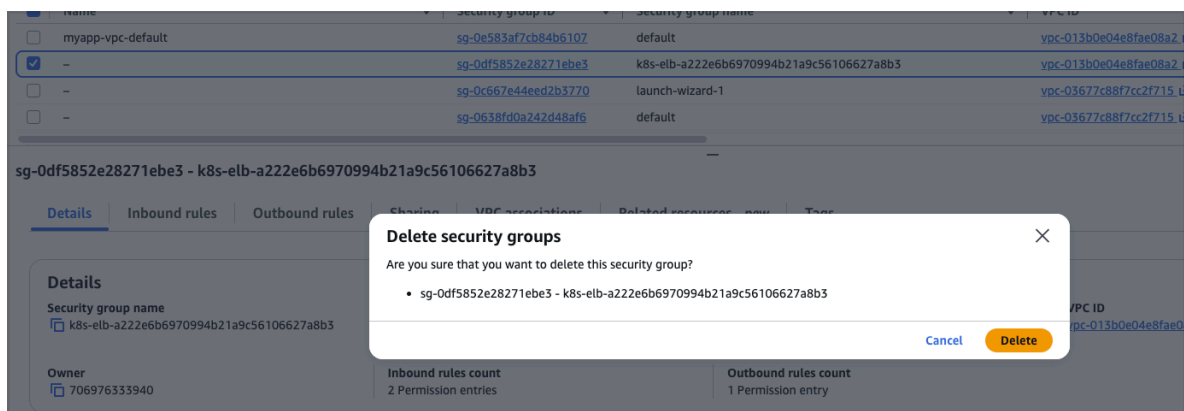
```
Error: deleting EC2 VPC (vpc-013b0e04e8fae08a2): operation error EC2: DeleteVpc, https response error StatusCode: 400, RequestID: 3626b3a9-dd94-4592-8cee-56993f16c7e5, api error DependencyViolation: The vpc 'vpc-013b0e04e8fae08a2' has dependencies and cannot be deleted.
```

Issue:

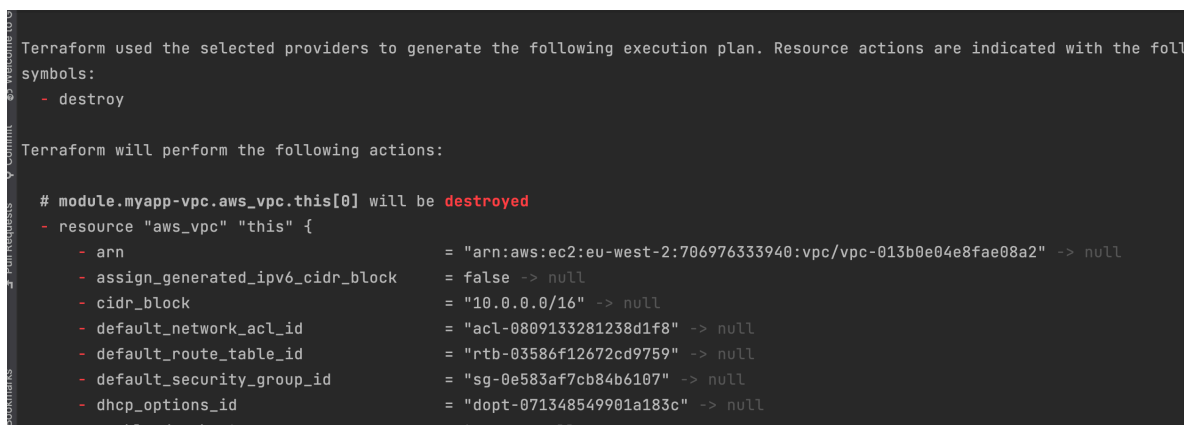
one leftover security group: sg-0df5852e28271ebe3 (k8s-elb-a222e6b6970994b21a9c56106627a8b3), the ELB's own security group. Same story as before — Kubernetes created it via the in-tree AWS cloud provider, so Terraform has no record of it and can't delete it. Since we already deleted the ELB itself, nothing should reference this SG anymore, so it's safe to remove.



So I delete it



-> terraform destroy



```
Plan: 0 to add, 0 to change, 1 to destroy.
```

```
Do you really want to destroy all resources?
```

```
Terraform will destroy all your managed infrastructure, as shown above.  
There is no undo. Only 'yes' will be accepted to confirm.
```

```
Enter a value: yes
```

```
module.myapp-vpc.aws_vpc.this[0]: Destroying... [id=vpc-013b0e04e8fae08a2]
```

```
module.myapp-vpc.aws_vpc.this[0]: Destruction complete after 1s
```

```
Destroy complete! Resources: 1 destroyed.
```

Now everything is destroyed! 